

Lab 06 · Los viajes al archivador

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

75 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

El problema N+1, medido en pantalla: **201 consultas y 71 ms para una pantalla que se resuelve con 1 consulta y 10 ms**. Y las tres formas de arreglarlo, con el criterio para elegir cuál.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. Los viajes al archivador, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · El crimen, medido	3
3.2. Paso 2 · JOIN FETCH: traerlo todo de una vez	4
3.3. Paso 3 · @EntityGraph: lo mismo, sin escribir JPQL	5
3.4. Paso 4 · Proyección: no traer lo que no se enseña	6
3.5. Paso 5 · La trampa	8
4. Lo que aprendiste	9
5. Para profundizar	9
6. Antes de cerrar	9

1 Antes de empezar

1.1 Qué vas a lograr

En el Lab 05 contaste 1 consulta con LAZY y 4 con EAGER, sobre seis trámites. Hoy son **200 contribuyentes y 1.000 trámites**, y ese «4» se convierte en **201**.

Vas a ver el número en pantalla, vas a arreglarlo de **tres formas distintas**, y vas a aprender cuándo se usa cada una. Y en el último paso vas a ver la trampa: **la solución del paso 2 es la peor opción para la pantalla del paso 5**.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 04 y 05 hechos	Sabes qué es LAZY y qué es una consulta derivada	Imprescindible
Estar en la carpeta del lab	cd labs/lab-06- rendimiento/practica	El cd no da error

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en labs/lab-06-rendimiento/solucion/.

1.4 La puesta a punto

```
cd labs/lab-06-rendimiento/practica
./mvnw spring-boot:run
```

Escucha en el **8089** y su PostgreSQL en el **55436**. **Párala con Ctrl+C**; los dos errores de arranque del Lab 04 salen aquí igual, con estos números.

Este lab siembra 200 contribuyentes y 1.000 trámites al arrancar, así que el primer arranque tarda algo más. Es lo que hace que los números se vean.

2 El caso

El archivador de la DGT ya funciona. Ahora hay que enseñar una pantalla con **la lista de contribuyentes y cuántos trámites tiene cada uno**. Doscientos contribuyentes en la lista.

2.1 Los viajes al archivador, que es la metáfora de este laboratorio

El archivador está en el sótano, y cada consulta es un viaje.

El archivista sube con la lista de los 200 contribuyentes: **un viaje**. Y entonces, para cada uno, alguien pregunta «¿y cuántos trámites tiene?» — y el archivista **vuelve a bajar**. Doscientas veces.

Total: **201 viajes** para una pantalla.

Ninguno de los viajes está mal. El archivista hace exactamente lo que se le pide, y cada bajada es rápida. **El problema es la cuenta**, y no se ve leyendo el código: el código dice `contribuyente.getTramites().size()`, que parece gratis.

Este laboratorio va de aprender a **contar los viajes** y de las tres formas de bajarlos a uno.

3 Los pasos

3.1 Paso 1 · El crimen, medido

Qué vamos a hacer

Correr la pantalla tal cual y **contar las consultas**.

Para entenderlo mejor

Poner un contador en la escalera del sótano. Nadie discute cuántos viajes hay cuando están contados.

El problema

El N+1 no se ve leyendo. El código es corto, claro y correcto; el desastre está en cuántas veces se ejecuta una línea que parece inocente. **Por eso lo primero no es arreglarlo: es medirlo.**

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

```
=== 1 · EL CRIMEN · findAll() y tocar la relación ===  
200 contribuyentes · 1000 trámites  
CONSULTAS: 201 · TIEMPO: 71 ms
```

201. Una por la lista, y **doscientas más**, una por contribuyente, disparadas por tocar `getTramites()` de cada uno.

Los milisegundos van a ser distintos en tu máquina, y da igual: lo que importa es **el número de consultas**, que es el mismo en todas partes. El tiempo se enseña sólo para que se vea que 201 viajes tampoco son gratis.

Vas bien si...

Ves **CONSULTAS: 201** en la primera demo. Si vieras otro número, comprueba que el `@ManyToOne` de `Tramite` sigue en **LAZY**.

Si te atascas

1 · EL PUERTO 55436 YA ESTA OCUPADO O ESTE MISMO PROYECTO YA ESTA CORRIENDO

Los dos candados del Lab 04, con los números de este lab. El propio mensaje trae el comando:

```
lsof -ti:55436 | xargs kill -9
```

2 · Las consultas salen 1 en vez de 201.

Alguien dejó un `JOIN FETCH` puesto, o el `fetch` de la relación está en **EAGER** con un solo contribuyente. Comprueba que estás en la demo 1.

3.2 Paso 2 · JOIN FETCH: traerlo todo de una vez

Qué vamos a hacer

Pedir los contribuyentes **y sus trámites** en una sola consulta.

Para entenderlo mejor

Decirle al archivista: «*baja una vez, y súbeme las fichas de los contribuyentes **con** las de sus trámites*». Un viaje, la carretilla más llena.

El problema

El **LAZY** del Lab 05 está bien como valor por defecto —no traer lo que no se pide—, pero aquí **sí se va a pedir, y para todos**. Hay que poder decirlo.

La alternativa, y por qué no

- **Cambiar la relación a EAGER:** arregla esta pantalla y rompe todas las demás, porque a partir de ahí **siempre** se traen los trámites, los mire quien los mire. La decisión se toma en la entidad, que es el sitio equivocado: quien sabe qué hace falta es **la consulta**, no la clase.
- **JOIN FETCH**, que es lo de aquí: la decisión se toma **por consulta**. Esta trae los trámites; las demás, no.

Se pega

En practica/src/main/java/cl/dgt/rendimiento/repositories/ContribuyenteRepository.java:

```
@Query("select distinct c from Contribuyente c left join fetch c.tramites")
List<Contribuyente> conJoinFetch();
```

Ese distinct no es decorativo. Sin él, un contribuyente con cinco trámites **sale cinco veces**: el join multiplica las filas y JPA te devuelve el mismo objeto repetido.

Lo que vas a ver

```
=== 2 · JOIN FETCH · traerlo todo de una vez ===
200 contribuyentes · 1000 trámites
CONSULTAS: 1 · TIEMPO: 17 ms
```

De 201 a 1. De 71 ms a 17.

Vas bien si...

CONSULTAS: 1, y los 200 contribuyentes siguen siendo 200 — no 1.000. Si salieran más de 200, te falta el distinct.

Si te atascas

1 · Salen 1.000 contribuyentes en vez de 200.

Falta el distinct. El join multiplicó las filas.

2 · QuerySyntaxException o un error de JPQL al arrancar.

En JPQL se escriben **nombres de clases y de campos**, no de tablas y columnas: Contribuyente c, c.tramites. Si escribiste contribuyente o tramite, no lo encuentra.

3.3 Paso 3 · @EntityGraph: lo mismo, sin escribir JPQL

Qué vamos a hacer

Conseguir el mismo resultado declarando **qué traer**, en vez de escribir la consulta.

Para entenderlo mejor

En vez de dictarle al archivista la frase entera, dejarle una nota: «*cuando traigas contribuyentes, tráete también los trámites*».

El problema

JOIN FETCH funciona y te obliga a escribir la consulta entera aunque lo único que quisieras decir fuera «tráete también esto». Con tres relaciones, la consulta se hace larga.

La alternativa, y por qué no

Las dos hacen lo mismo y el SQL que sale es equivalente. La elección es de legibilidad:

- **JOIN FETCH** cuando ya estás escribiendo la consulta por otro motivo —hay where, hay order by—: añadir el fetch ahí es natural.
- **@EntityGraph** cuando la consulta es «tráemelo todo» y lo único que quieres declarar es qué acompaña. Es más corto y no obliga a nadie a leer JPQL.

Se pega

```
@EntityGraph(attributePaths = "tramites")
List<Contribuyente> findAllBy();
```

Ese findAllBy con By final y nada detrás no es un error de escritura. Es la forma que tiene Spring Data de decir «todos, sin condición»: findAll a secas es el método heredado, y no admite anotaciones propias.

Lo que vas a ver

```
=== 3 · @EntityGraph · lo mismo, sin JPQL ===
200 contribuyentes · 1000 trámites
CONSULTAS: 1 · TIEMPO: 21 ms
```

Vas bien si...

CONSULTAS: 1, igual que con JOIN FETCH. Las dos formas llegan al mismo sitio.

Si te atascas

1 · **Cannot find method findAllBy** o vuelven 201 consultas.

Escribiste findAll en vez de findAllBy. Sobre el findAll heredado, @EntityGraph no aplica.

3.4 Paso 4 · Proyección: no traer lo que no se enseña

Qué vamos a hacer

Traer **sólo las tres columnas que la pantalla muestra**, en vez de los objetos completos.

Para entenderlo mejor

La pantalla enseña RUT, razón social y **un número**. No enseña los trámites: enseña **cuántos**. Subir las 1.000 fichas de trámite para contarlas y tirarlas es cargar la carretilla para nada.

El problema

JOIN FETCH y @EntityGraph arreglaron el número de viajes, pero **siguen subiendo todo**: los 1.000 trámites llegan enteros a la memoria de la aplicación para que después alguien haga `.size()`.

La alternativa, y por qué no

- **Entidades completas** (pasos 2 y 3): necesarias si vas a **modificar** algo, porque una entidad gestionada es lo que Hibernate vigila.
- **Proyección a un record**, que es lo de aquí: para una pantalla de sólo lectura, trae lo justo y el conteo lo hace la base. Lo que vuelve **no es una entidad** y no se puede modificar — que para una consulta es una ventaja, no una limitación.

Se pega

El DTO, archivo **nuevo** practica/src/main/java/cl/dgt/rendimiento/dto/ResumenContribuyente.java:

```
package cl.dgt.rendimiento.dto;

public record ResumenContribuyente(String rut, String razonSocial, long
↪ cuantosTramites) {
}
```

Y la consulta, en el repositorio:

```
@Query("""
    select new cl.dgt.rendimiento.dto.ResumenContribuyente(c.rut, c.razonSocial,
↪ count(t))
    from Contribuyente c
    left join c.tramites t
    group by c.id, c.rut, c.razonSocial
    order by c.id
    """)
List<ResumenContribuyente> resumen();
```

El `new cl.dgt.rendimiento.dto.ResumenContribuyente(...)` dentro del JPQL es lo que construye el record directamente desde la consulta. Va con el paquete completo: JPQL no tiene import.

Lo que vas a ver

```
=== 4 · PROYECCIÓN · traer solo lo que se muestra ===
200 contribuyentes · 1000 trámites
CONSULTAS: 1 · TIEMPO: 10 ms
primera fila -> ResumenContribuyente[rut=71.001.007-1, razonSocial=Contribuyente
↪ 001 Ltda., cuantosTramites=5]
```

1 consulta y 10 ms. El conteo lo hizo PostgreSQL, que para eso está.

Vas bien si...

CONSULTAS: 1 y el tiempo **por debajo** del de los pasos 2 y 3. Y lo que vuelve es un Resumen-Contribuyente, no un Contribuyente.

Si te atascas

1 · Unable to locate appropriate constructor

La lista de argumentos del `new` en la consulta no coincide con el constructor del `record`: ni en número, ni en orden, ni en tipos. `count(t)` devuelve `long`.

2 · Falta el paquete completo en el `new`.

JPQL no sabe de `import`: hay que escribir `cl.dgt.rendimiento.dto.ResumenContribuyente` entero.

3.5 Paso 5 · La trampa

Qué vamos a hacer

Otra pantalla, que **sólo muestra razones sociales**, y ver por qué la solución del paso 2 sería la peor aquí.

Para entenderlo mejor

Esta pantalla no menciona los trámites. Si el archivista sube igualmente las 1.000 fichas de trámite «por si acaso», está haciendo el trabajo del paso 1 al revés: **un solo viaje, pero con la carretilla llena de cosas que nadie va a mirar**.

El problema

Una vez que alguien descubre `JOIN FETCH`, la tentación es ponerlo en todas partes. Y `JOIN FETCH` **siempre** trae la relación, la necesites o no.

La alternativa, y por qué no

No hay una técnica que gane siempre, y ése es el contenido del paso:

Lo que hace la pantalla	Lo que conviene
Lee entidades y las modifica	Entidades, con <code>JOIN FETCH</code> o <code>@EntityGraph</code>
Lee y muestra campos de la relación	<code>JOIN FETCH</code> o <code>@EntityGraph</code>
Lee y muestra un agregado (un conteo, una suma)	Proyección
No toca la relación	<code>findAll()</code> a secas — sin fetch de nada

Lo que vas a ver

```
=== 5 · LA OTRA PANTALLA · solo razones sociales ===
200 contribuyentes · 4600 letras en total
CONSULTAS: 1      · TIEMPO: 3 ms
```


3 ms, el más rápido de las cinco demos, y con la consulta más tonta de todas: la que no trae nada que no se use.

Vas bien si...

La demo 5 es más rápida que la 2 y la 3, y hace lo mismo: una sola consulta. La diferencia es lo que **no** trae.

4 Lo que aprendiste

1 · El N+1 no se lee: se cuenta.

`contribuyente.getTramites().size()` parece gratis y cuesta una consulta por contribuyente. La única forma de verlo es contar, y por eso este lab trae un contador puesto.

2 · Hay tres formas de arreglarlo, y no son intercambiables.

`JOIN FETCH` cuando ya escribes la consulta. `@EntityGraph` cuando sólo quieres declarar qué acompaña. **Proyección** cuando la pantalla muestra un agregado y no va a modificar nada.

3 · La decisión de qué traer es de la consulta, no de la entidad.

Poner `EAGER` en la relación arregla una pantalla y encarece todas las demás. Por eso el valor por defecto es `LAZY` y el fetch se pide donde hace falta.

4 · La técnica correcta depende de la pantalla.

Un `JOIN FETCH` en una pantalla que no mira la relación es un N+1 al revés: un viaje, pero cargado de lo que nadie va a usar. No hay una respuesta única, y por eso el paso 5 existe.

5 Para profundizar

- Pon `@ManyToOne(fetch = EAGER)` en `Tramite` y vuelve a correr las cinco demos. ¿Cuáles empeoran?
- Quita el `distinct` del `JOIN FETCH` y cuenta los contribuyentes que vuelven.
- Escribe una proyección con la **SUMA** de algo en vez del conteo, y compara el tiempo.
- Sube la siembra a **2.000 contribuyentes** en `CargadorDeDatos` y vuelve a medir. ¿Crece el tiempo del paso 1 igual que el del paso 4?

6 Antes de cerrar

Párala con `Ctrl+C`.

```
./mvnw clean
```

Lo que te llevas:

Tocar una relación `LAZY` dentro de un bucle son N consultas más. Se arregla por consulta —con `JOIN FETCH`, con `@EntityGraph` o con una proyección—, nunca cambiando la entidad. Y la técnica buena depende de qué muestra la pantalla.

Lo que queda pendiente, y abre el Lab 07: hoy todo lo que hiciste fue **leer**. Cuando dos usuarios escriben **a la vez** sobre la misma fila, aparecen problemas que ninguna consulta arregla. En el Lab 07 se emiten veinte folios a la vez y se cuenta cuántos sobreviven.